



Tec Gurus
SOMOS Y FORMAMOS EXPERTOS EN T.I.

CURSO

Linux Desde Cero

CLASE 4

Regex, red y copias remotas

www.tecgurus.net

INSTRUCTOR

Ing. Abimael Domínguez



AGENDA • 09:00–14:00

Clase 4 — Regex, red y copias remotas

Archivo: capítulo 11 completo.

HORA	ACTIVIDAD
09:00–09:15	Verificar datos, scripts y acceso remoto.
09:15–09:50	Regex sobre logs.
09:50–10:20	IP, rutas, puertos y DNS.
10:20–11:00	SSH, claves y huellas.
11:00–11:30	Receso.
11:30–12:15	SCP/SFTP en ambos sentidos y hashes.
12:15–12:45	Compilación, linkado y Makefile de Pac-Man.
12:45–13:40	Reto 11: entrega DevOps.
13:40–13:55	Diagnóstico, seguridad y Telnet/FTP como contexto.
13:55–14:00	Limpieza de recursos y cierre.

No recortar: regex básica, SSH, SCP y verificación.

Recortar primero: ejecución del juego y detalles avanzados de DNS.

Índice interactivo

Selecciona un apartado para ir directamente a su contenido. En cada encabezado encontrarás el enlace para volver aquí.

CAPÍTULO 11 COMPLETO	
Compilación, regex, red y copias remotas	
Panorama de la sesión	→
Índice	→
Objetivos	→
Antes de empezar	→
11.1 Compilación en Linux	→
11.2 Linkado	→
11.3 make	→
11.4 Búsqueda avanzada y regex	→
11.5 Caracteres especiales	→
11.6 Expresiones de un carácter	→
11.7 Expresiones generales	→
• Práctica regex resuelta	→
11.8 Comandos útiles de red	→
11.9 Protocolos de Internet	→
11.10 Nombres y DNS	→
11.11 Telnet, FTP, SSH, SCP y SFTP	→
• Conexión SSH	→
• Copias con SCP	→
• SFTP	→
Práctica guiada resuelta — Entrega remota	→
Errores frecuentes	→
Reto 11 — Entrega DevOps	→
• Criterios de comprobación	→
Checklist	→

11

ENTREGA REPRODUCIBLE

Compilación, regex, red y copias remotas

Del diagnóstico de logs y red a una transferencia segura, verificable y reproducible mediante SSH.

Capítulo 11

regex

SSH · SCP · SFTP

sha256sum

Panorama de la sesión

[Índice ↑](#)

El nombre comercial del tema es “SCP Copias Remotas”, pero el temario agrupa compilación, expresiones regulares y redes. Este capítulo los conecta mediante una entrega reproducible.

Índice

[Índice ↑](#)

- [Objetivos](#)
- [Antes de empezar](#)
- [11.1 Compilación en Linux](#)
- [11.2 Linkado](#)
- [11.3 `make`](#)
- [11.4 Búsqueda avanzada y regex](#)
- [11.5–11.7 Expresiones regulares](#)
- [11.8–11.10 Red, protocolos y DNS](#)
- [11.11 SSH, SCP y SFTP](#)
- [Práctica guiada: entrega remota](#)
- [Errores frecuentes](#)
- [Reto 11](#)

Objetivos

[Índice ↑](#)

- Distinguir código fuente, objeto, librería y ejecutable.
- Automatizar un build con `make`.
- Extraer información de logs mediante regex.
- Diagnosticar IP, puertos y DNS.
- Conectarse y transferir archivos con SSH, SCP y SFTP.

Antes de empezar

[Índice ↑](#)

Abre una terminal en la raíz del curso y guarda esa ruta en una variable:

```
cd ~/linux-desde-cero
RAIZ_CURSO=$(pwd)
printf 'Raíz del curso: %s\n' "$RAIZ_CURSO"
```

`RAIZ_CURSO` permitirá volver al repositorio después de entrar a la carpeta de Pac-Man. Si cierras la terminal, ejecuta nuevamente estas tres líneas.

En este capítulo se crearán artefactos únicamente dentro de `src/11-scp-copias-remotas/pacman-game/` y `laboratorio/`.

Continuidad entre sesiones. Antes de compilar o buscar logs, confirma los recursos versionados:

`ls data/dummy_logs.txt src/11-scp-copias-remotas/pacman-game/pacman.c`. Si alguno falta, no crees un archivo vacío ni descargues un reemplazo improvisado; vuelve a la raíz del curso y recupera el repositorio. Los artefactos de compilación y las carpetas de `laboratorio/` sí se crean durante esta clase.

11.1 Compilación en Linux

[Índice ↑](#)

El recurso **Pac-Man** permite observar el build sin aprender C.

Usaremos estos nombres:

Archivo	Significado
<code>pacman.c</code>	código fuente legible
<code>pacman.o</code>	objeto producido por la compilación
<code>pacman_game</code>	ejecutable producido por el linkado
<code>Makefile</code>	reglas para automatizar el build

La primera línea entra a la carpeta del proyecto. Los comandos siguientes se ejecutan desde ahí.

```
cd src/11-scp-copias-remotas/pacman-game
sudo apt update
sudo apt install -y build-essential libncurses-dev
gcc -Wall -Wextra -Wpedantic -std=c11 -O2 \
  -c pacman.c -o pacman.o
file pacman.c pacman.o
```

SALIDA REPRESENTATIVA

```
pacman.c: C source, Unicode text
pacman.o: ELF 64-bit LSB relocatable, x86-64
```

- `-c` : compila sin enlazar.
- `-o` : nombre de salida.
- `-Wall -Wextra -Wpedantic` : habilita diagnósticos útiles.
- `-O2` : optimización moderada.
- `pacman.o` contiene código objeto, todavía no es un programa ejecutable.

`sudo apt install` instala compilador, `make` y headers de ncurses. Si ya están presentes, APT informa que están en su versión actual y no los duplica.

11.2 Linkado

[Índice ↑](#)

TERMINAL · BASH

```
gcc pacman.o -o pacman_game -lncurses
file pacman_game
ldd pacman_game | grep ncurses
./pacman_game
```

- El linker resuelve símbolos y crea el ejecutable.
- `-lncurses` solicita la librería `libncurses`.
- `ldd` muestra bibliotecas dinámicas; no debe usarse sobre binarios no confiables.
- Dentro del juego, usa flechas y `q` para salir.

11.3 make

[Índice ↑](#)

El `Makefile` declara objetivos, dependencias y recetas:

```
make clean
make compilar
make enlazar
make run
make clean
```

SALIDA REPRESENTATIVA

```
gcc ... -c pacman.c -o pacman.o
gcc pacman.o -o pacman_game -lnurses
```

- `make` ejecuta sólo lo necesario según dependencias y fechas.
- Una receta debe comenzar con tabulación.
- `.PHONY` marca nombres que no representan archivos.
- `clean` retira artefactos; el código fuente permanece.

Después de `make clean`, vuelve a la raíz antes de trabajar con `data/`:

```
cd "$RAIZ_CURSO"
test -f data/dummy_logs.txt && echo "De vuelta en la raíz del curso"
```

Si omities este paso, `grep data/dummy_logs.txt` fallará porque `data/` no existe dentro de `pacman-game`.

11.4 Búsqueda avanzada y regex

[Índice ↑](#)

Una expresión regular describe patrones de texto. Para logs se usarán expresiones extendidas con `grep -E`.

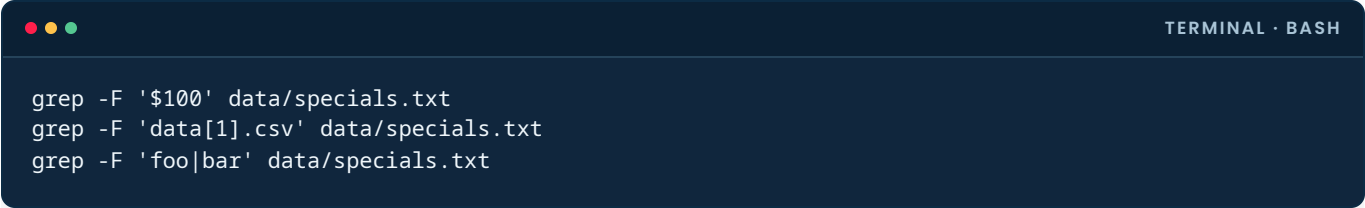
```
grep -En 'WARN|ERROR' data/dummy_logs.txt
grep -E '^([0-9]{4}-[0-9]{2}-[0-9]{2})' data/dummy_logs.txt
grep -oE 'User [[:alnum:]]_-' data/dummy_logs.txt
```

- `^/$`: inicio/final.
- `[0-9]`: un dígito.
- `{4}`: repetición exacta.
- `|`: alternancia.
- `-o`: imprime sólo la coincidencia.
- `[[:alnum:]]`: clase portable de letras y números.

11.5 Caracteres especiales

[Índice ↑](#)

Regex y shell tienen metacaracteres distintos. Las comillas simples evitan que el shell transforme el patrón:



```
grep -F '$100' data/specials.txt
grep -F 'data[1].csv' data/specials.txt
grep -F 'foo|bar' data/specials.txt
```

`grep -F` trata el patrón literalmente; es preferible cuando no se necesita regex.

11.6 Expresiones de un carácter

[Índice ↑](#)

TERMINAL · BASH

```
grep -En 'User [[:alnum:]]_-' data/dummy_logs.txt
grep -En 'CPU|Disk|Network' data/dummy_logs.txt
grep -En '[0-9]{2}:[0-9]{2}:[0-9]{2}' data/dummy_logs.txt
```

`.` significa cualquier carácter; úsalo sólo cuando esa amplitud sea intencional. Para un punto literal usa `\.`

11.7 Expresiones generales

[Índice ↑](#)

Práctica regex resuelta

[Índice ↑](#)

En este punto debes estar en la raíz del curso. El bloque crea dos archivos:

- `incidentes.txt`: líneas completas que contienen `WARN` o `ERROR` ;
- `usuarios.txt`: únicamente nombres de usuario, ordenados y sin duplicados.

```
mkdir -p laboratorio/red
grep -En 'WARN|ERROR' data/dummy_logs.txt \
| tee laboratorio/red/incidentes.txt

grep -oE 'User [[:alnum:]]_+' data/dummy_logs.txt \
| cut -d' ' -f2 \
| sort -u \
> laboratorio/red/usuarios.txt

printf 'Incidentes=%s Usuarios=%s\n' \
"$(wc -l < laboratorio/red/incidentes.txt)" \
"$(wc -l < laboratorio/red/usuarios.txt)"
```

La práctica separa extracción, normalización, ordenamiento y reporte para poder depurar cada etapa.

Con los fixtures actuales, la última línea debe mostrar `Incidentes=3 Usuarios=3`.

11.8 Comandos útiles de red

[Índice ↑](#)

Prepara las herramientas que no estén presentes en una instalación mínima:

```
sudo apt update
sudo apt install -y iproute2 iputils-ping curl dnsutils openssh-client
```

```
ip -brief address
ip route
ss -tuln
ping -c 3 1.1.1.1
curl -I https://example.com
```

No necesitas reemplazar valores en este bloque. `1.1.1.1` y `example.com` son destinos públicos usados sólo para comprobar conectividad; si una red bloquea ICMP, `ping` puede fallar aunque `curl` funcione.

- `ip -brief address`: interfaces y direcciones resumidas.
- `ip route`: rutas; la línea `default` indica gateway.
- `ss -tuln`: sockets TCP/UDP en escucha, sin resolver nombres.
- `ping -c 3`: tres pruebas ICMP; un bloqueo de ICMP no demuestra por sí solo que el host esté caído.
- `curl -I`: cabeceras HTTP.

`ifconfig` y `netstat` son referencias históricas; para nuevas prácticas se prefieren `ip` y `ss`.

11.9 Protocolos de Internet

[Índice ↑](#)

- **IP:** direccionamiento y encaminamiento.
- **TCP:** flujo confiable orientado a conexión; SSH usa normalmente TCP 22.
- **UDP:** datagramas sin conexión; DNS puede usar UDP o TCP.
- **Puerto:** identifica un servicio dentro de un host.

TERMINAL · BASH

```
getent services ssh  
ss -tn state established
```

No confundas “puerto abierto” con “acceso autorizado”: todavía existen autenticación y políticas.

11.10 Nombres y DNS

[Índice ↑](#)

TERMINAL · BASH

```
getent hosts example.com  
dig example.com A +short  
cat /etc/resolv.conf
```

- `getent hosts`: usa la resolución configurada por el sistema, incluyendo `/etc/hosts` y DNS.
- `dig`: consulta DNS con detalle.
- `/etc/resolv.conf`: configuración efectiva; puede estar administrada por otra herramienta.

Diagnóstico gradual:

1. ¿La interfaz tiene IP?
2. ¿Existe ruta por defecto?
3. ¿Resuelve el nombre?
4. ¿Responde el puerto/protocolo esperado?
5. ¿La aplicación autentica y responde correctamente?

11.11 Telnet, FTP, SSH, SCP y SFTP

[Índice ↑](#)

Telnet y FTP transmiten autenticación/datos sin la protección esperada actualmente. Se estudian sólo como contexto. Para acceso y transferencia se usan SSH, SCP o SFTP.

Conexión SSH

[Índice ↑](#)

Hay tres datos que cambian para cada alumno:

Dato	Valor usado en el curso	De dónde sale
Usuario remoto	<code>ubuntu</code>	usuario predeterminado de la AMI Ubuntu
Clave privada	<code>~/.ssh/curso-linux.pem</code>	archivo descargado al crear EC2
IP pública	diferente por instancia	consola EC2, columna Public IPv4 address

SINTAXIS GENERAL

TERMINAL · BASH

```
chmod 400 <ruta_clave_privada>
ssh -i <ruta_clave_privada> <usuario>@<IP_PUBLICA>
```

No copies literalmente los marcadores `<...>`. Para evitar repetir valores, define variables. Sustituye sólo la IP del ejemplo:

TERMINAL · BASH

```
CLAVE="$HOME/.ssh/curso-linux.pem"
USUARIO_REMOTO="ubuntu"
IP_PUBLICA="203.0.113.10" # Sustituye por la IP real de tu EC2

chmod 400 "$CLAVE"
ssh -i "$CLAVE" "${USUARIO_REMOTO}@${IP_PUBLICA}"
```

`203.0.113.10` es una IP reservada para documentación y no corresponde a tu servidor. Después de asignar tu IP real, el resto del capítulo se puede copiar sin volver a editarla.

Opciones importantes:

- `-i`: archivo de identidad.
- `-p`: puerto SSH cuando no es 22.
- `-v`: diagnóstico; puede repetirse hasta `-vvv`.

En `scp`, el puerto se indica con `-P` mayúscula; `-p` minúscula significa preservar tiempos y modos.

La clave privada nunca se copia al servidor. El servidor almacena la clave pública autorizada.

Copias con SCP

[Índice ↑](#)

Los comandos siguientes usan `CLAVE`, `USUARIO_REMOTO` e `IP_PUBLICA` definidos en el apartado anterior. Si abriste una terminal nueva, vuelve a definirlos.

```
terminal-bash$ scp -i "$CLAVE" \
laboratorio/red/incidentes.txt \
"${USUARIO_REMOTO}@${IP_PUBLICA}:/home/ubuntu/"

terminal-bash$ scp -i "$CLAVE" \
"${USUARIO_REMOTO}@${IP_PUBLICA}:/home/ubuntu/incidentes.txt" \
laboratorio/red/incidentes-remoto.txt

terminal-bash$ sha256sum laboratorio/red/incidentes*.txt
```

- Sintaxis remota: `[usuario@]host:ruta`.
- `-r`: copia directorios; para entregas se prefiere empaquetar y verificar.
- `sha256sum`: confirma integridad, no autenticidad del origen.

SFTP

[Índice ↑](#)

```
terminal-bash$ sftp -i "$CLAVE" "${USUARIO_REMOTO}@${IP_PUBLICA}"
```

Dentro de SFTP: `pwd` / `ls` operan en remoto; `lpwd` / `lls` en local; `put` sube, `get` descarga y `bye` sale.

Práctica guiada resuelta — Entrega remota

[Índice ↑](#)

Continuidad dentro de la clase. Esta entrega necesita `laboratorio/red/incidentes.txt` y `laboratorio/red/usuarios.txt`. Ambos se crean en la práctica 11.7. Si faltan, vuelve a esa práctica y ejecútala completa; no crees archivos vacíos para que SCP continúe. Si abriste una terminal nueva, vuelve a definir `CLAVE`, `USUARIO_REMOTO` e `IP_PUBLICA` antes de copiar.

Requisitos antes de copiar:

```
test -s laboratorio/red/incidentes.txt \
&& test -s laboratorio/red/usuarios.txt \
&& echo "Reportes listos"
```

Si no aparece `Reportes listos`, completa primero la práctica 11.7. El siguiente bloque empaqueta esos dos archivos, genera el hash, sube ambos a `/home/ubuntu` y verifica el paquete en EC2.

```
mkdir -p laboratorio/entrega
cp laboratorio/red/incidentes.txt laboratorio/entrega/
cp laboratorio/red/usuarios.txt laboratorio/entrega/

tar -czf laboratorio/entrega-linux.tar.gz -C laboratorio entrega
tar -tzf laboratorio/entrega-linux.tar.gz
(cd laboratorio && \
  sha256sum entrega-linux.tar.gz | tee entrega-linux.tar.gz.sha256)

scp -i "$CLAVE" \
  laboratorio/entrega-linux.tar.gz* \
  "${USUARIO_REMOTO}@${IP_PUBLICA}:/home/ubuntu/"

ssh -i "$CLAVE" "${USUARIO_REMOTO}@${IP_PUBLICA}" \
  'sha256sum -c entrega-linux.tar.gz.sha256'
```

Salida principal:

```
entrega-linux.tar.gz: OK
```

El hash se calcula localmente y se verifica remotamente. La comilla simple hace que el comando entre comillas se interprete en el host remoto.

Errores frecuentes

[Índice ↑](#)

- Copiar o publicar la clave privada.
- Abrir SSH a todo Internet sin necesidad.
- Aceptar una huella cambiada sin investigar.
- Usar Telnet/FTP para credenciales.
- Probar sólo `ping` y concluir que toda la aplicación funciona.
- Escribir regex demasiado amplia y confiar sin revisar muestras.

Reto 11 — Entrega DevOps

[Índice ↑](#)

Ver respuesta

Trabaja dentro de `laboratorio/reto11`. Genera `incidentes.txt` con regex y `red.txt` con diagnóstico no sensible. Empaqueta la carpeta como `laboratorio/reto11.tar.gz`, calcula su hash, transfiere paquete y hash a EC2 y verifica allí su integridad. Recupera después `incidentes.txt` como `laboratorio/incidentes-recuperado.txt`.

Criterios de comprobación

[Índice ↑](#)

- El reporte se genera a partir de datos, no se edita manualmente.
- El paquete se lista antes de enviarse.
- La verificación remota muestra `OK`.
- Se demuestra transferencia en ambos sentidos.
- No se copia ni imprime la clave privada.

Checklist

[Índice ↑](#)

- ☐ Distingo fuente, objeto, librería y ejecutable.
- ☐ Puedo ejecutar y limpiar un build con `make`.
- ☐ Extraigo información de logs con regex controladas.
- ☐ Diagnostico IP, ruta, DNS, puerto y aplicación por capas.
- ☐ Uso SSH/SCP/SFTP y verifico integridad.



EVIDENCIA Y CIERRE

Clase 4 completada

Al terminar la Clase 4, conserva el paquete transferido y la evidencia de su verificación remota.



Extraigo incidentes y usuarios de los logs con expresiones regulares controladas.



Diagnostico IP, ruta, DNS, puerto y aplicación por capas.



Transfiero el paquete en ambos sentidos y verifico su hash remotamente.

CURSO COMPLETADO · LINUX DESDE CERO